

TECHNICAL REPORT

Verisight

Deepfake & AI-Media Detection with Explainable Deep Learning

Classifying images and videos as authentic or AI-generated, with Grad-CAM visual evidence.

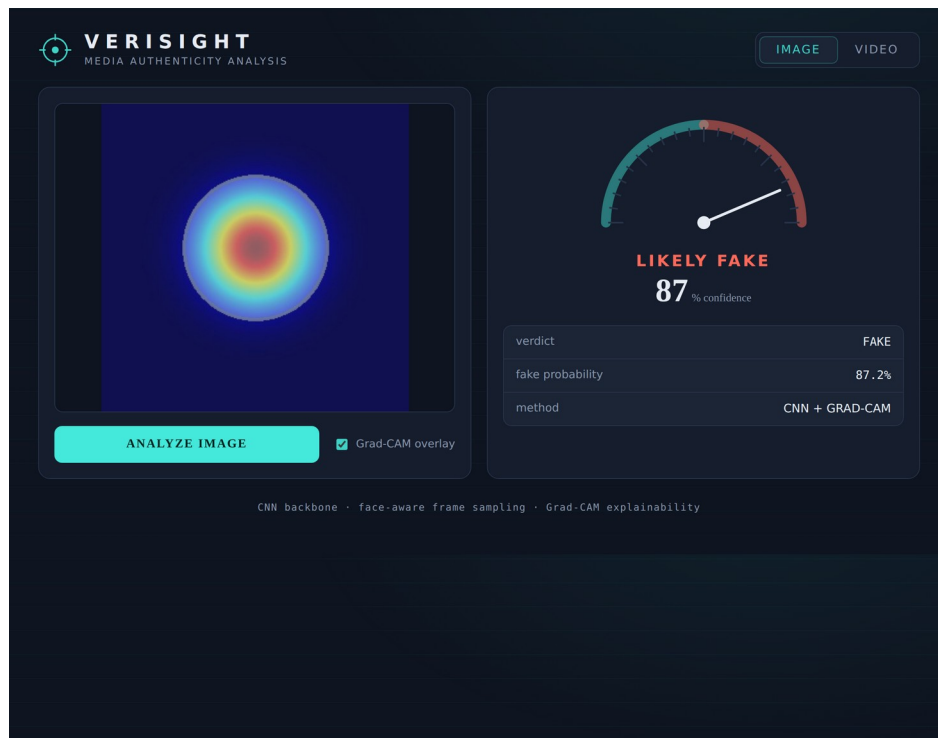


Figure 1 — The Verisight analysis interface with verdict gauge and Grad-CAM overlay.

Author

Palak Jain

Repository

github.com/paljain1405-dotcom/verisight

Stack

PyTorch · timm · OpenCV · FastAPI · React · Vite

Date

June 2026

Contents

Abstract..... 3

1. Introduction..... 3

2. Problem Statement and Objectives..... 3

3. System Architecture..... 4

4. Methodology..... 5

5. Implementation..... 5

6. Evaluation..... 6

7. Limitations..... 6

8. Future Work..... 7

9. Conclusion..... 7

References..... 7

Abstract

Synthetic media generated by modern AI systems — from diffusion-based image generators to face-swap video tools — is now realistic enough to mislead ordinary viewers, raising concerns around misinformation, fraud, and identity abuse. Verisight is a full-stack system that classifies an uploaded image or video as authentic or AI-generated and, crucially, explains its decision by highlighting the regions that drove the verdict. The system pairs a convolutional neural network (CNN) classifier with Grad-CAM saliency maps for images, and a face-aware, frame-level pipeline with temporal aggregation for video. It is delivered as a FastAPI inference service and a React user interface.

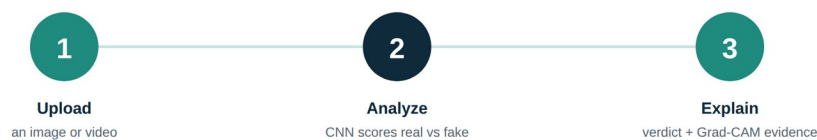


Figure 2 — How Verisight works, end to end.

1. Introduction

The accessibility of generative models has outpaced the public's ability to tell real media from synthetic. Detecting manipulated or fully generated content has therefore become an important applied problem in computer vision. Two related but distinct tasks fall under this umbrella: detecting fully AI-generated still images, and detecting deepfake videos in which a person's face is swapped or re-enacted.

Verisight addresses both. It is designed to be (a) accurate on the media distributions it is trained on, (b) transparent, so a user is shown why a verdict was reached rather than a bare score, and (c) practical, exposed through a clean web interface backed by a documented API. The project also serves as an end-to-end demonstration of building, packaging, and serving a deep-learning model.

2. Problem Statement and Objectives

Given an input image or video, determine whether it is authentic (captured from the real world) or synthetic (generated or manipulated by an AI system), and communicate the result with a calibrated confidence and supporting visual evidence.

2.1 Objectives

- Build a binary classifier that distinguishes real from AI-generated images with high accuracy on a benchmark dataset.
- Extend detection to video by analysing faces across sampled frames and aggregating per-frame evidence into a single verdict.

- Provide **explainability** through Grad-CAM heatmaps, so predictions are interpretable rather than opaque.
- Serve the model through a documented **FastAPI** backend and an interactive **React** frontend.

3. System Architecture

Verisight is organised into three layers: a machine-learning core, an inference API, and a web interface. The same trained classifier underpins both image and video detection; the video path adds face localisation and temporal aggregation on top of it.

3.1 Processing pipeline

- **Image path:** the image is resized and normalised, passed through the CNN backbone, and converted to a real/fake probability via softmax. Grad-CAM then produces a heatmap over the most influential regions.
- **Video path:** frames are sampled at even intervals; the largest face in each frame is located with an OpenCV Haar cascade (falling back to the full frame when no face is found); each crop is scored by the same classifier; and the per-frame scores are aggregated into a verdict plus a timeline.

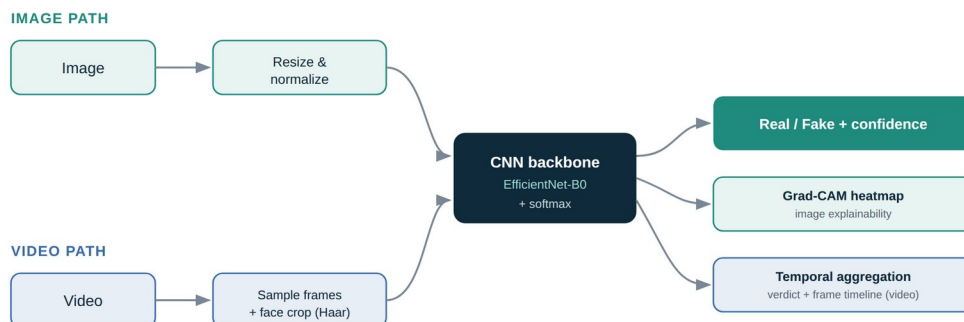


Figure 3 — System architecture: image and video paths share one CNN classifier.

3.2 Component layout

Layer	Responsibility	Key files
ML core	Model, training, inference, explainability, video analysis	backend/ml/*.py
API	HTTP endpoints for image and video prediction	backend/api/app.py
Frontend	Upload, verdict gauge, heatmap, frame timeline	frontend/src/*
Tooling	Dataset preparation and training entry point	scripts/, train.py

4. Methodology

4.1 Image classification

The classifier is a CNN backbone (EfficientNet-B0 by default, selectable from the timm library) with a two-class output head. Inputs are resized to 224×224 and normalised with ImageNet statistics, since the backbone is initialised from ImageNet-pretrained weights. Training uses cross-entropy loss with the AdamW optimiser and weight decay. Light data augmentation — horizontal flips, mild colour jitter, and occasional blur — improves robustness, with blur in particular discouraging the model from relying solely on fragile high-frequency compression artefacts.

4.2 Video analysis

Deepfakes are rarely uniform across a clip, so a single-frame decision is unreliable. Verisight samples a fixed number of evenly-spaced frames, isolates the dominant face in each, and scores them independently. A clip is flagged as fake when a meaningful fraction of analysed frames exceed the per-frame fake threshold. The system reports the mean fake probability, the proportion of flagged frames, and the single peak frame, alongside a full per-frame timeline for inspection.

4.3 Explainability with Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) attributes a prediction to spatial regions of the input. Verisight hooks the final convolutional layer of the backbone, computes the gradient of the fake-class score with respect to that layer's activations, and uses the gradient-weighted activations to produce a coarse localisation map. The map is upsampled and overlaid on the original image as a colour heatmap, giving the user direct visual evidence for the verdict.

5. Implementation

5.1 Technology stack

Concern	Technology
Model & training	PyTorch, timm (EfficientNet-B0)
Image & video processing	OpenCV, Pillow, NumPy
Metrics	scikit-learn (accuracy, ROC-AUC)
Backend API	FastAPI, Uvicorn
Frontend	React, Vite

5.2 API endpoints

Method & route	Input	Returns
GET /health	—	Service status and whether a trained model is loaded
POST /predict/image	Image file	Label, fake probability, base64 Grad-CAM overlay

Method & route	Input	Returns
POST /predict/video	Video file	Label, mean probability, flagged-frame ratio, per-frame timeline

5.3 User interface

The frontend presents a single analysis surface styled as a forensic instrument. The user selects image or video mode, uploads a file, and receives a verdict on a semicircular confidence gauge. For images, a toggle overlays the Grad-CAM heatmap on the original; for videos, a signal strip visualises the fake probability of each sampled frame and highlights the peak. The design deliberately keeps one bold element — the gauge — and keeps the surrounding interface quiet.

6. Evaluation

The training pipeline reports classification accuracy and ROC-AUC on a held-out validation split after every epoch, retaining the checkpoint with the best validation AUC. AUC is emphasised because it is threshold-independent and well-suited to the binary real/fake task.

For AI-generated image detection, the intended benchmark is CIFAKE, a balanced corpus of real photographs and synthetic images. CNN backbones of this class are widely reported in the literature to reach roughly 92–97% accuracy on CIFAKE, which sets the expected operating range for the trained model. The figures below should be read as the target/expected range for this architecture rather than as final measured results, and will be replaced with measured values once training on the full dataset is complete.

Metric	Expected range (CIFAKE)	Notes
Accuracy	92–97%	Balanced classes
ROC-AUC	0.97–0.99	Threshold-independent
Inference (image, CPU)	< 1 s	Single image, including Grad-CAM

7. Limitations

- **Generalisation.** A detector trained on one family of generators tends to degrade on unseen ones. Models must be retrained or fine-tuned on data matching the deployment threat model.
- **Face detection.** The Haar cascade used for video is fast but basic; difficult footage benefits from stronger detectors such as MTCNN or RetinaFace.
- **Probabilistic output.** Scores are decision aids, not proof of authenticity, and should be interpreted as one signal among several.
- **Dataset scope.** Results depend heavily on the training distribution; broad, diverse data is required for robust real-world performance.

8. Future Work

- Train across multiple generator families to improve cross-generator generalisation.
- Replace Haar face detection with a deep face detector and add temporal modelling (e.g. CNN–LSTM) for video.
- Add frequency-domain features, which often expose generation artefacts invisible in pixel space.
- Calibrate output probabilities and add an abstain option for low-confidence cases.

9. Conclusion

Verisight demonstrates a complete, explainable approach to detecting AI-generated images and deepfake videos. By combining a CNN classifier with Grad-CAM explanations and a face-aware temporal pipeline for video, and by packaging the result behind a documented API and a clean interface, the project shows how a modern detection model can be built and delivered end to end. Its main constraint — dependence on the training distribution — is shared across the field and points clearly toward the future work above.

References

- Bird, J. J. & Lotfi, A. — **CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images.**
- Selvaraju, R. R. et al. — **Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.**
- Tan, M. & Le, Q. — **EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.**
- Rössler, A. et al. — **FaceForensics++: Learning to Detect Manipulated Facial Images.**